Nand2Tetris

SET-A & SET-B

Complete Exam Solutions

Exam-ready answers • Truth tables • Circuit diagrams • HDL code

Prepared from the two uploaded question papers

How to use these solutions

For 5- and 10-mark questions, write the definition or working, draw the circuit, include the truth table or calculation where relevant, and finish with the HDL code.

Note: The diagrams are simplified text diagrams for easy reproduction in a handwritten exam. HDL examples assume the standard Nand2Tetris chips referenced in each implementation are available.

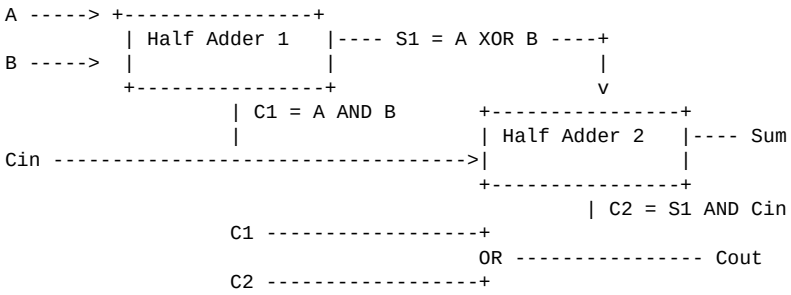
SET-A — Solutions

Section 1: One-word answers

Q	Question	Answer
1	8-bit 2's complement of –42	11010110
2	NOT(X AND Y) = NOT(X) OR NOT(Y)	De Morgan's law
3	Routes several inputs to one output	Multiplexer (MUX)
4	Hack ALU flag for negative output	ng
5	Address bits required by RAM64	6 bits

Q1. Full Adder using two Half Adders and one OR gate (5 marks)

A Full Adder adds three 1-bit inputs A, B and Cin, producing Sum and Cout.



Boolean expressions:

S1 = A XOR B; C1 = A·B; Sum = S1 XOR Cin; C2 = S1·Cin; Cout = C1 + C2.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

```
CHIP FullAdder {
    IN a, b, c;
    OUT sum, carry;

    PARTS:
    HalfAdder(a=a, b=b, sum=s1, carry=c1);
    HalfAdder(a=s1, b=c, sum=sum, carry=c2);
    Or(a=c1, b=c2, out=carry);
}
```

Q2. Why NAND is functionally complete (5 marks)

A NAND gate is functionally complete because NOT, AND and OR can all be built from NAND gates alone; these operations can express any Boolean function.

NOT using NAND

Y = NOT(A AND A) = NOT A. Connect both NAND inputs to A.

AND using NAND

n = NAND(A,B); Y = NAND(n,n) = A AND B.

OR using NAND

By De Morgan's law, A OR B = NAND(NOT A, NOT B). Use NAND(A,A) and NAND(B,B) to generate the inverted inputs.

A	B	NAND	AND	OR
0	0	1	0	0
0	1	1	0	1
1	0	1	0	1
1	1	0	1	1

```
CHIP NotUsingNand {
    IN in;
    OUT out;
    PARTS:
    Nand(a=in, b=in, out=out);
}
```

```
CHIP AndUsingNand {
    IN a, b;
    OUT out;
    PARTS:
    Nand(a=a, b=b, out=n);
    Nand(a=n, b=n, out=out);
}
```

```
CHIP OrUsingNand {
    IN a, b;
    OUT out;
    PARTS:
    Nand(a=a, b=a, out=na);
    Nand(a=b, b=b, out=nb);
    Nand(a=na, b=nb, out=out);
}
```

Q3(a). Mux4Way (5 marks)

A 4-way multiplexer selects one of four inputs a, b, c or d using two select bits.

```
a ----+
b ----| MUX 4-WAY |---- out
c ----|           |
d ----+---- sel[1:0]
```

sel[1]	sel[0]	Output
0	0	a
0	1	b
1	0	c
1	1	d

out = (NOT s1 AND NOT s0 AND a) OR (NOT s1 AND s0 AND b) OR (s1 AND NOT s0 AND c) OR (s1 AND s0 AND d).

```
CHIP Mux4Way {
```

```
    IN a, b, c, d, sel[2];
    OUT out;
    PARTS:
    Mux(a=a, b=b, sel=sel[0], out=m1);
    Mux(a=c, b=d, sel=sel[0], out=m2);
    Mux(a=m1, b=m2, sel=sel[1], out=out);
}
```

Q3(b). RAM64 address 100101

RAM64 uses 6 address bits because 2^6 = 64. Split address 100101 into upper bits 100 (decimal 4) and lower bits 101 (decimal 5). With zero-based numbering, select RAM8 chip 4 (the fifth chip), register 5 (the sixth register).

• DMux: uses the upper address bits to select which RAM8 receives the write-enable signal.

• Mux: selects the output of the appropriate RAM8 for reading.

• The lower three address bits select the register within the chosen RAM8.

```
Address[5:3] ---> DMux ---> select one of 8 RAM8 chips
Address[2:0] -----------------> register address within selected RAM8
RAM8 outputs -----------------> Mux ---> RAM64 output
```

Q4. Hack ALU trace (5 marks)

Controls: zx=1, nx=1, zy=0, ny=0, f=1, no=1. Choose x=0000000000000000 and y=0000000000000001.

Stage	Control	Operation/result
1	zx=1	x becomes 0000000000000000
2	nx=1	x becomes 1111111111111111
3	zy=0	y remains 0000000000000001
4	ny=0	y remains 0000000000000001
5	f=1	Add: (-1) + 1 = 0
6	no=1	Invert zero: 1111111111111111

Final: out=1111111111111111; zr=0 because output is not zero; ng=1 because the most significant bit is 1.

```
x -> zeroing (zx) -> inversion (nx) --+
                                      +-> AND/ADD (f) -> inversion (no) -> out
y -> zeroing (zy) -> inversion (ny) ------+
                                          |-> zr, ng flags
```

Q5. Hack Program Counter (5 marks)

The PC is a 16-bit sequential chip holding the address of the next instruction.

```
in[16] ----------------------------+
                                   v
out[16] -> Inc16 -> MUX(hold/inc) -> MUX(load) -> MUX(reset) -> Register -> out[16]
                                    ^             ^              ^
                                   inc           load           reset
```

reset	load	inc	Operation
1	X	X	Reset to zero
0	1	X	Load input
0	0	1	Increment
0	0	0	Hold

Priority: reset > load > increment > hold. If all three controls are asserted, output becomes zero.

```
CHIP PC {
    IN in[16], load, inc, reset;
    OUT out[16];
    PARTS:
    Inc16(in=out, out=incremented);
    Mux16(a=out, b=incremented, sel=inc, out=step1);
```

```
    Mux16(a=step1, b=in, sel=load, out=step2);
    Mux16(a=step2, b=false, sel=reset, out=next);
    Register(in=next, load=true, out=out);
}
```

SET-B — Solutions

Section 1: One-mark answers

Q	Question	Answer
1	Boolean algebra: a + 1	1
2	HDL keyword for chip inputs	IN
3	Selects one of two inputs using a control bit	Mux
4	Hack ALU flag when all output bits are zero	zr
5	Adds two single bits without carry-in	Half Adder

Q1. Design a 16-bit adder (5 marks)

A 16-bit adder adds two 16-bit binary inputs and returns a 16-bit result. The final carry beyond bit 15 is discarded.

```
a[16] ----+
          +---- Add16 ---- out[16]
b[16] ----+
```

```
CHIP Adder16 {
    IN a[16], b[16];
    OUT out[16];
    PARTS:
    Add16(a=a, b=b, out=out);
}
```

Example: 0000000000000101 (5) + 0000000000000011 (3) = 0000000000001000 (8).

If implementing the carry chain manually, use a HalfAdder for bit 0 and FullAdder chips for bits 1 through 15, passing each carry to the next stage. The final carry is discarded.

```
CHIP Adder16 {
    IN a[16], b[16];
    OUT out[16];
    PARTS:
    HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c1);
    FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
    FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
    FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
    FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
    FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
    FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
    FullAdder(a=a[7], b=b[7], c=c7, sum=out[7], carry=c8);
    FullAdder(a=a[8], b=b[8], c=c8, sum=out[8], carry=c9);
    FullAdder(a=a[9], b=b[9], c=c9, sum=out[9], carry=c10);
    FullAdder(a=a[10], b=b[10], c=c10, sum=out[10], carry=c11);
    FullAdder(a=a[11], b=b[11], c=c11, sum=out[11], carry=c12);
    FullAdder(a=a[12], b=b[12], c=c12, sum=out[12], carry=c13);
    FullAdder(a=a[13], b=b[13], c=c13, sum=out[13], carry=c14);
    FullAdder(a=a[14], b=b[14], c=c14, sum=out[14], carry=c15);
    FullAdder(a=a[15], b=b[15], c=c15, sum=out[15], carry=discard);
}
```

Q2. Two-input XOR gate (5 marks)

XOR outputs 1 when the two inputs differ. Boolean expression: Y = A XOR B = (NOT A AND B) OR (A AND NOT B).

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

```
A ----+
      XOR ---- Y
B ----+
```

```
CHIP Xor {
    IN a, b;
    OUT out;
    PARTS:
    Nand(a=a, b=b, out=n1);
    Nand(a=a, b=n1, out=n2);
    Nand(a=b, b=n1, out=n3);
    Nand(a=n2, b=n3, out=out);
}
```

Verification: inputs 00 -> 0; 01 -> 1; 10 -> 1; 11 -> 0.

Q3(a). Demultiplexer / DMux (10-mark question, part a)

A DMux routes a single input to one of two outputs based on a select bit.

```
                +---------+----> a
in ------------->|  DMux   |
sel ------------>|         +----> b
                +---------+
```

in	sel	a	b
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

a = in AND NOT(sel); b = in AND sel.

```
CHIP DMux {
    IN in, sel;
    OUT a, b;
    PARTS:
    Not(in=sel, out=nsel);
    And(a=in, b=nsel, out=a);
    And(a=in, b=sel, out=b);
}
```

Q3(b). D Flip-Flop and BIT chip (10-mark question, part b)

A D flip-flop stores one bit and captures the D input at the active clock edge. The Q output holds the stored value between clock edges.

```
D --------> +---------+
CLK -------->|   DFF   |-------> Q
             +---------+
```

A BIT chip adds a load-controlled MUX: when load=0 it feeds the previous output back to the DFF; when load=1 it feeds the new input.

```
in ----+
       MUX ----> DFF ----> out
out ---+  ^
          load
```

```
CHIP Bit {
    IN in, load;
    OUT out;
    PARTS:
    Mux(a=out, b=in, sel=load, out=muxOut);
    DFF(in=muxOut, out=out);
}
```

load	Next stored value
0	Previous value retained
1	Input stored at next clock edge

Q4(a). Hack ALU trace (10-mark question, part a)

Controls: zx=1, nx=1, zy=0, ny=1, f=1, no=0. Choose x=0000000000000000 and y=0000000000000001.

Stage	Control	Operation/result
1	zx=1	x = 0000000000000000
2	nx=1	x = 1111111111111111
3	zy=0	y = 0000000000000001
4	ny=1	y = 1111111111111110
5	f=1	Add processed inputs
6	no=0	Keep sum unchanged

16-bit addition: 1111111111111111 + 1111111111111110 = 1 1111111111111101. Discard the carry beyond 16 bits.

Final: out=1111111111111101 (–3); zr=0; ng=1.

Q4(b). Program Counter (10-mark question, part b)

The PC is a 16-bit register storing the next instruction address.

reset	load	inc	Operation
0	0	0	Hold
0	0	1	Increment
0	1	0	Load
0	1	1	Load
1	0	0	Reset
1	0	1	Reset
1	1	0	Reset
1	1	1	Reset

Priority: reset > load > increment > hold.

```
CHIP PC {
    IN in[16], load, inc, reset;
    OUT out[16];
    PARTS:
    Inc16(in=out, out=incremented);
    Mux16(a=out, b=incremented, sel=inc, out=step1);
    Mux16(a=step1, b=in, sel=load, out=step2);
    Mux16(a=step2, b=false, sel=reset, out=next);
    Register(in=next, load=true, out=out);
}
```

Last-minute revision sheet

Topic	Remember
Boolean algebra	a+1=1; NAND is functionally complete; De Morgan's laws
Half Adder	Sum=A XOR B; Carry=A AND B
Full Adder	Sum=A XOR B XOR Cin; Cout=AB + Cin(A XOR B)
MUX / DMux	MUX selects one of many inputs; DMux routes one input to an output
RAM64	6 address bits; 8 RAM8 chips; upper 3 bits choose chip, lower 3 choose register
Hack ALU	f=1 means ADD; f=0 means AND; zr=1 for zero; ng=1 for negative sign bit
DFF / BIT	DFF stores one bit; BIT adds load control using a feedback MUX
PC priority	reset > load > increment > hold

HDL reminder: chip declarations use IN and OUT; internal wiring goes under PARTS:. Check chip names and pin names against the standard Nand2Tetris library.